CMSC 603 High-Performance Distributed Systems

CUDA memory access pattern

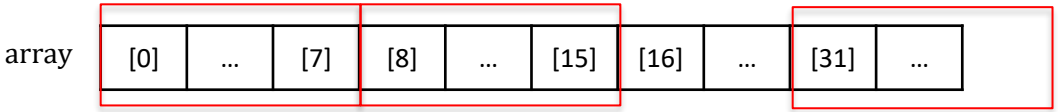


Dr. Preetam Ghosh
Professor
Department of Computer Science
pghosh@vcu.edu

Memory access pattern on CPUs and GPUs

- Data is generally loaded / stored in blocks of transactions of 32/64/128 bytes

```
for (int i = 0; i < size; i++)
    array[i] = i;
```

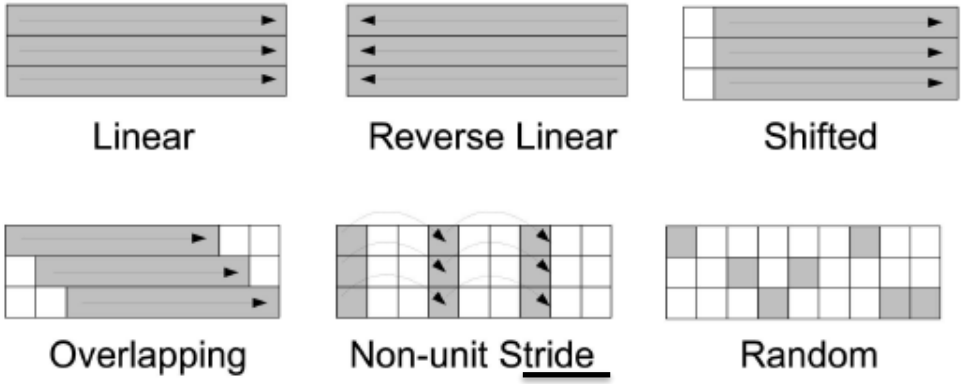


- A single transaction will cache contiguous values used in iterations of the loop:

  To use array[0], first execute a transaction of 32 bytes retrieving array[0-7]
  Now iterate the loop using retrieved values for array[0-7] already cached
  To continue using array[8], then execute a transaction of 32 bytes retrieving array[8-15]
  Now iterate the loop using retrieved values for array[8-15] already cached

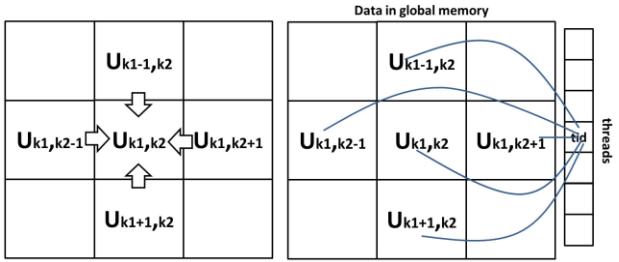
- The goal is to exploit the **spatial and temporal locality** of the data

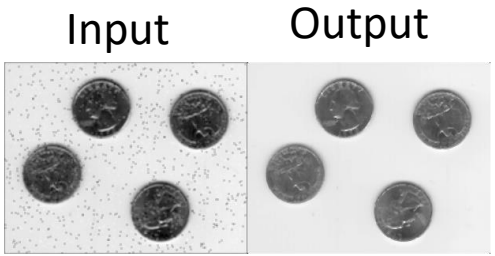
Memory access pattern on CPUs and GPUs





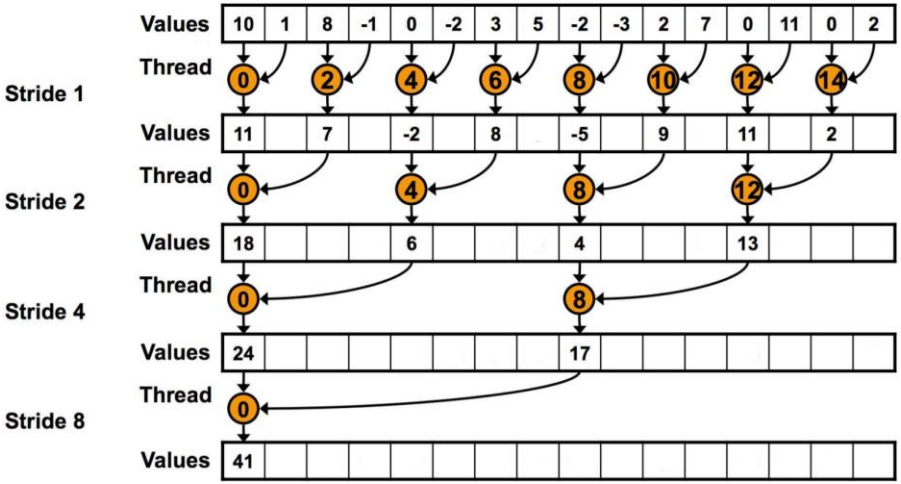
Example: memory accesses in image denoise with a median filter

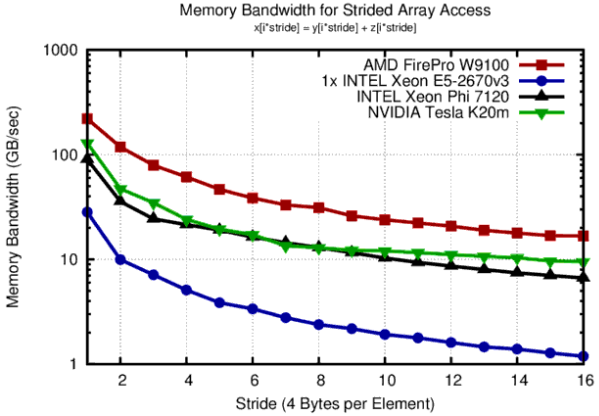




Stride in memory access

- Stride introduces a gap between effective memory accesses
- Stride and effective bandwidth
- **Example**: parallel reduction







The importance of the memory hierarchy and the memory access pattern

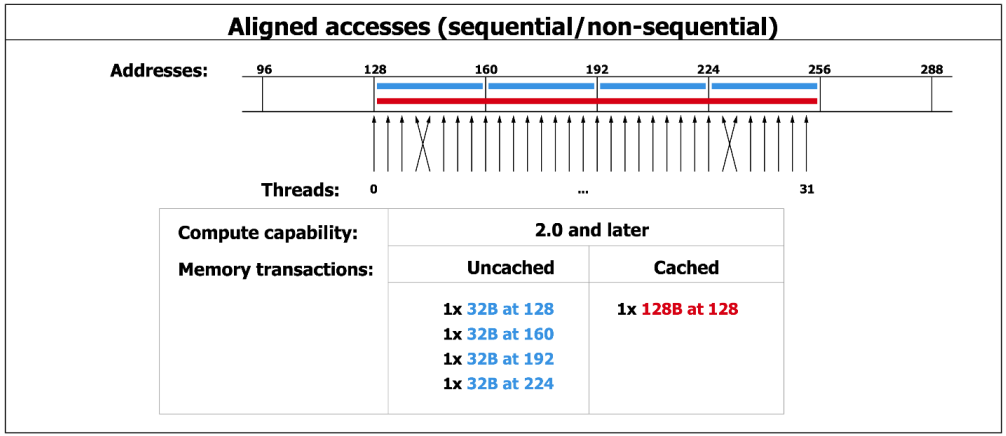
- **Local** memory: register memory with no latency
- **Shared** memory latency ~5ns (max 164KB per block)
- **Global** memory latency ~300 ns (GPU memory 8-24 GB)
- GPU arithmetic instruction latency ~10 ns
- Having many threads allows hiding the latency of memory accesses by interleaving warps in a multiprocessor's execution
- Memory access patterns that play nicely are called **coalesced memory accesses**
- Memory accesses are done **per warp** in large groups as memory transactions
- Coalesced memory accesses minimize the number of transactions

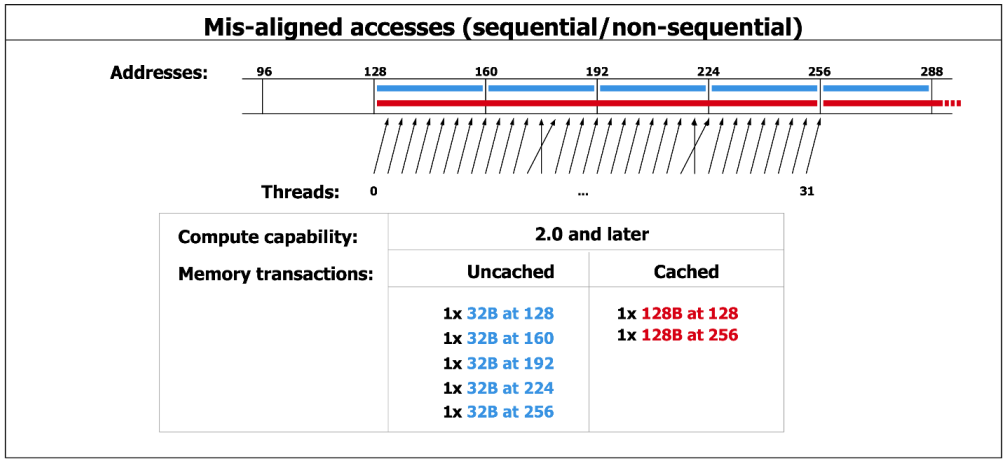
Memory coalescing

Alignment of memory accesses vs effective transactions needed

Memory transactions goals:

- minimize their number
- maximize their size
- minimize data waste





Advanced indexing using a multi-dimensional grid of multi-dimensional blocks

- Example: sum two 2D matrixes

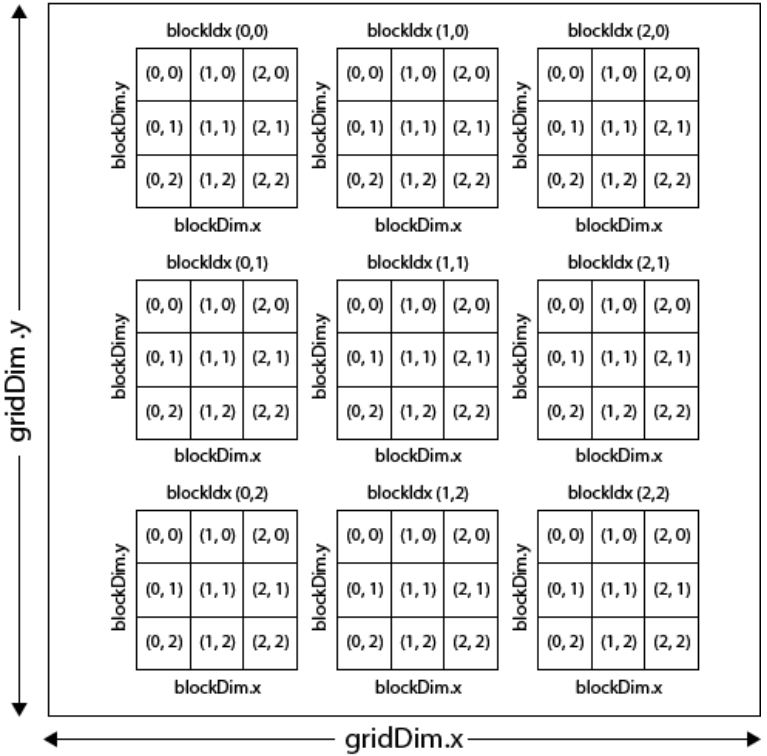
Linear indexing of a matrix:

[0][0] -> [0][N] -> [1][0] -> [1][N] -> [N][1] -> [N][N]

*int column = (blockDim.x * blockIdx.x) + threadIdx.x;*
*int row = (blockDim.y * blockIdx.y) + threadIdx.y;*

Coalesced memory access pattern indexing
*int tid = (blockDim.x * gridDim.x * row) + column;* ✓

What if?
Non-coalesced memory access pattern indexing
*int tid = (blockDim.y * gridDim.y * column) + row;* ✗



Advanced indexing using a multi-dimensional grid of multi-dimensional blocks

- Option 1: naïve definition of a 2D grid of 2D blocks

```
int threadsPerBlockDim = 16;
int gridDimSize = (matrixSize + threadsPerBlockDim - 1) / threadsPerBlockDim;

dim3 blockSize(threadsPerBlockDim, threadsPerBlockDim);
dim3 gridSize (gridDimSize, gridDimSize);

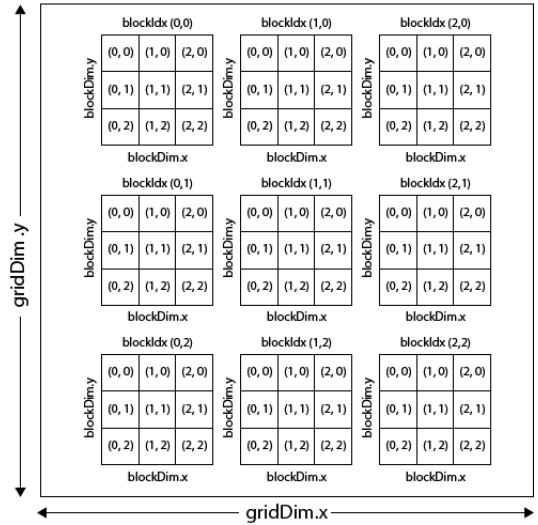
matrixAdd<<<gridSize, blockSize>>>(d_A, d_B, d_C, numElements);
```

Advantages

- Easy to understand
- Logical model matches human-understanding
- Divide problem into smaller 2D submatrixes

Disadvantages:

- Divisibility of the blocks in X and Y
- Not the best memory access pattern



Advanced indexing using a multi-dimensional grid of multi-dimensional blocks

- Option 2: worst-ever approach, using a 1D grid and 1D-y block

```
int threadsPerBlock = 256;
int gridDim = (numElements + threadsPerBlock - 1) / threadsPerBlock;

dim3 blocksize(1, threadsPerBlock);

matrixAdd<<<gridDim, blocksize>>>(d_A, d_B, d_C, numElements);
```

Considering an equal number of threads per block

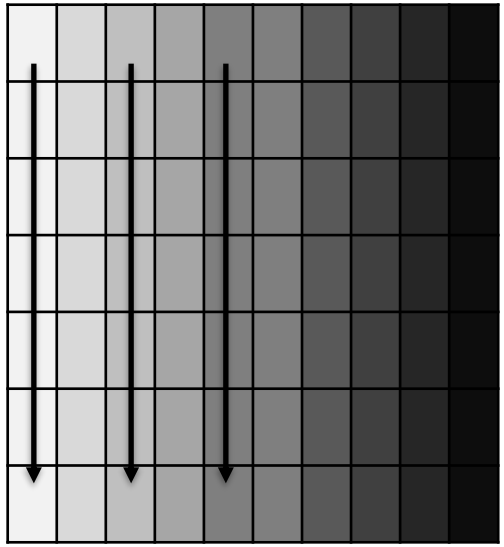
Advantages

- Divisibility of the blocks and Y

Disadvantages:

- Terrible memory access pattern!
- Stride = matrix width
- Huge performance bottleneck!!!

Threads



Advanced indexing using a multi-dimensional grid of multi-dimensional blocks

- Option 3: best approach, using a 1D grid and 1D-x block

```
int threadsPerBlock = 256;
int gridDim = (numElements + threadsPerBlock - 1) / threadsPerBlock;

matrixAdd<<<gridDim, threadsPerBlock>>>(d_A, d_B, d_C, numElements);
```

Considering an equal number of threads per block



Advantages

- Minimum loss due to divisibility of the blocks 1D, only up to 1 block extra
- Best access pattern, stride = 1, fully coalesced memory
- Matrix width doesn't affect the performance!

Disadvantages: none

Measuring elapsed time: events

- CUDA kernels are **non-blocking** calls
- The code will continue until synchronization is forced by
  - cudaDeviceSynchronize()
  - cudaMemcpy()
  - cudaEventSynchronize()

```
float milliseconds;
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);

cudaEventRecord(start);

// do something!

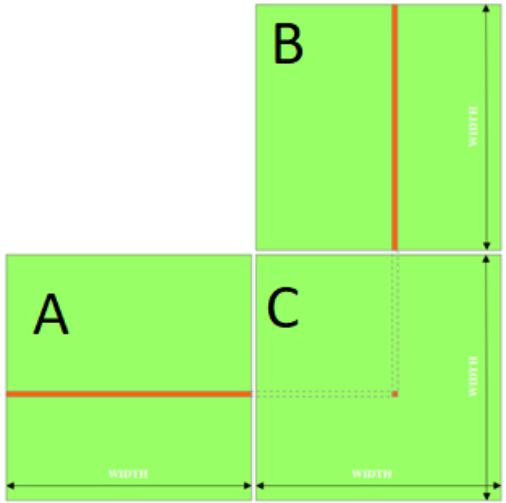
cudaEventRecord(stop);
cudaEventSynchronize(stop);
cudaEventElapsedTime(&milliseconds, start, stop);
printf("GPU time %f ms\n", milliseconds);
```

- **Example:** matrixAdd.cu

Matrix multiplication

- Classic academic example to show the benefits of shared memory
- C = A * B of size WIDTH x WIDTH
- CPU sequential implementation:

```
void MatrixMultiplicationHost (float* A, float* B, float* C, int width)
{
   for (int i = 0; i < width; i++) {
      for (int j = 0; j < width; j++) {
         float sum = 0;
         for (int k = 0; k < width; k++) {
            sum += A[i * width + k] * B[k * width + j];
         }
         C[i * width + j] = sum;
      }
   }
}
```



First approach for matrix multiplication on a GPU

- Each thread computes one element of the result matrix
- Each thread uses a row of M and a column of N
- Access pattern: consecutive threads access contiguous rows/cols?
- Inner loop iterates rows (vertically), leading to a bad memory access pattern!

```
__global__ void matrixMul(float *A, float *B, float *C, int width)
{
    int column = ( blockDim.x * blockIdx.x ) + threadIdx.x;
    int row    = ( blockDim.y * blockIdx.y ) + threadIdx.y;

    if (row < width && column < width)
    {
        float sum = 0;

        for(int k = 0; k < width; k++)
            sum += A[row * width + k] * B[k * width + column];

        C[row*width + column] = sum;
    }
}
```

- **Examples:** matrixMul.cu

CMSC 603 High-Performance Distributed Systems

CUDA memory access pattern



Dr. Preetam Ghosh
Professor
Department of Computer Science
pghosh@vcu.edu